

# Distributed Training

pig7selene

September 5, 2026

## Abstract

Large language models exceed the memory and throughput limits of a single accelerator, so pretraining must divide data, model state, and computation across coordinated ranks. This chapter derives gradient synchronization for Distributed Data Parallel, contrasts replication with sharding, and explains Tensor, Pipeline, and Sequence Parallelism. It then develops the memory logic of ZeRO and Fully Sharded Data Parallel, relates communication to realized scaling efficiency, and gives the contracts required to make a multidimensional parallel plan correct and recoverable.

## 1 Introduction

The pretraining budget in Chapter 9 is expressed in tokens and FLOPs, but that budget must be executed by an actual training system. A dense model may not fit in one accelerator’s memory once its parameters, gradients, optimizer state, and activations are present. Even when it does fit, one accelerator can make the planned token budget take an impractical amount of wall-clock time. Distributed training addresses both constraints by placing different pieces of a training step on multiple ranks.

The central difficulty is not merely placing tensors on more devices. All ranks must evaluate the same mathematical objective, keep the intended parameter state coherent, and communicate the information that a local computation no longer possesses. Parallelism therefore has two distinct purposes. **Data parallelism** replicates a model and divides examples across replicas. **Model parallelism** divides the model computation or state itself. The right system usually combines them because a training job needs both more aggregate throughput and less per-rank state.

This chapter builds on the token-level objective in Chapter 5, optimizer state in Chapter 7, numerical state in Chapter 8, and the compute-versus-wall-time distinction in Chapter 9. It does not prescribe a vendor-specific runtime, network topology, or inference-serving stack.

## 2 Parallelism, Ranks, and Batches

Let  $W$  be the total number of ranks. A rank is one independently scheduled participant in a distributed process group; it need not be identified with a particular physical device for the mathematical discussion. A **replicated** tensor has a full copy on every rank in a group. A **sharded** tensor is partitioned so that each rank holds only a disjoint piece, with an explicit rule for reconstructing or communicating the whole.

The local batch and global batch must also be separated. Let  $n_D$  be the Data Parallel degree, let  $b_{\text{mu}}$  be the microbatch size on each Data Parallel replica, and let  $M$  be the number of such microbatches accumulated into one optimizer step. In the simple equal-size case,

$$B_{\text{local}} = Mb_{\text{mu}}, \quad B_{\text{global}} = n_D Mb_{\text{mu}}. \quad (1)$$

Tensor, Pipeline, and Sequence Parallel ranks collaborate on the **same** local microbatch; they do not multiply the number of examples in Equation 1. Variable sequence lengths and loss masks require token-weighted accounting rather than this compact example, as Chapter 5 explains.

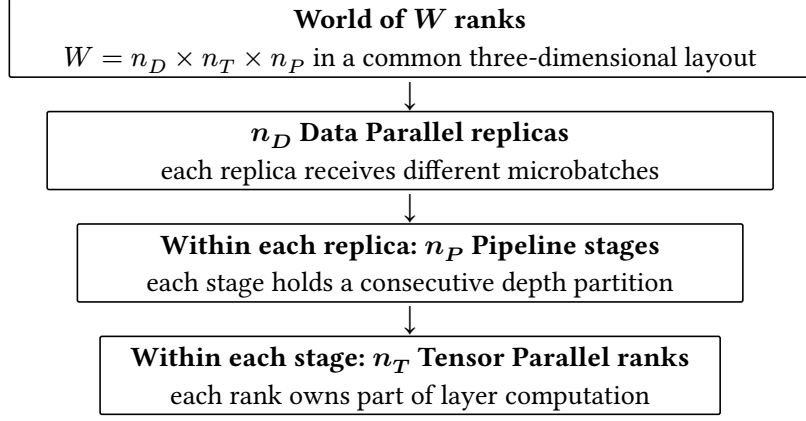


Figure 1: A common three-dimensional decomposition. Sequence Parallelism commonly reuses the Tensor Parallel group to partition selected activations, so it need not introduce another independent factor in world size.

Figure 1 is a logical hierarchy, not a claim that every process group is nested in exactly this order. It records a useful accounting rule: in a conventional three-dimensional layout,  $W = n_D n_T n_P$ , where  $n_T$  and  $n_P$  are the Tensor and Pipeline Parallel degrees. Sequence Parallelism often operates over the same  $n_T$  ranks rather than adding a fourth independent factor. Some systems add other dimensions, but each must specify which ranks share data, parameters, activations, and collectives.

### 3 Data Parallelism and Gradient Synchronization

In ordinary Data Parallelism, every rank begins a step with the same parameters  $\theta_t$  and receives a different local batch. Let  $g_r$  be rank  $r$ ’s local gradient for an equal-weight local objective. Distributed Data Parallel (DDP) computes the global gradient by an all-reduce average:

$$g = \frac{1}{n_D} \text{AllReduce}_{\text{sum}}(g_1, \dots, g_{n_D}) = \frac{1}{n_D} \sum_{r=1}^{n_D} g_r. \quad (2)$$

Every participant receives the same  $g$ , so applying the deterministic optimizer update from Chapter 7 produces the same  $\theta_{t+1}$  on every replica. This is the mathematical reason replicated DDP is coherent: independent forward and backward passes are permitted because their gradients are reconciled before the shared update. The DDP design and its overlap of gradient communication with backward computation are described by Li et al. [1].

For Causal Language Modeling, the local loss may contain a different number of valid target tokens due to packing or masking. Let  $S_r = \sum_{b,t} m_{b,t} \ell_{b,t}$  be a local loss numerator and let  $N_r = \sum_{b,t} m_{b,t}$  be its valid-token count. The globally token-averaged objective is

$$\mathcal{L}_{\text{global}} = \frac{\sum_{r=1}^{n_D} S_r}{\sum_{r=1}^{n_D} N_r}. \quad (3)$$

An unweighted average of local **mean** losses equals Equation 3 only when the  $N_r$  are equal. A distributed implementation must therefore equalize valid-token counts, scale each local loss against the global denominator, or explicitly all-reduce correctly weighted gradient numerators. More ranks do not excuse a changed objective.

#### 3.1 All-Reduce and the Replication Limit

All-reduce is a collective operation: it combines a tensor across a process group and makes the result available to every member. In DDP, the payload is usually the gradient buffer. A ring implementation of an all-reduce with payload  $q$  bytes moves approximately

$$q_{\text{ring}} \approx 2 \frac{n_D - 1}{n_D} q \quad (4)$$

bytes per rank, ignoring protocol overhead and using a common reduce-scatter plus all-gather decomposition. Other algorithms trade latency, bandwidth, and topology differently, but no all-reduce is free. The group must finish the necessary collective before its replicas can safely take the next synchronized update.

DDP reduces the data and activation work per rank as  $n_D$  grows, but it does not reduce the replicated training state. Let  $P$  be the declared parameter count, let  $b_\theta$ ,  $b_g$ , and  $b_{\text{opt}}$  be bytes per parameter for parameters, gradients, and total optimizer state, and let  $M_{\text{act}}$  be local activation memory. A schematic DDP memory model is

$$M_{\text{DDP}} \approx P(b_\theta + b_g + b_{\text{opt}}) + M_{\text{act}}. \quad (5)$$

Adding DDP ranks can shorten the wall-clock time of a feasible model, but every rank must still hold this full state. This is why ordinary DDP eventually fails to make a larger model trainable even when more accelerators are available: the model state is replicated rather than sharded.

## 4 Tensor Parallelism

Tensor Parallelism partitions a computation **inside** a Transformer layer across a small group of ranks. It is model parallelism, not a data-replication scheme. Consider a linear map  $Y = XW$  with  $X \in \mathbb{R}^{b \times d_{\text{in}}}$  and  $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ . If  $W$  is split by output columns across  $n_T$  ranks, then

$$W = [W_1, \dots, W_{n_T}], q \quad Y = [XW_1, \dots, XW_{n_T}]. \quad (6)$$

Each rank computes a different feature slice of  $Y$  locally. This is a column-parallel linear layer. If instead the input features and corresponding rows of  $W$  are split, each rank forms a partial result and the full output requires a sum:

$$Y = \sum_{j=1}^{n_T} X_j W_j. \quad (7)$$

The right side of Equation 7 is normally materialized by an all-reduce. Transformer MLPs can arrange the expansion projection as column-parallel, apply its activation locally, then arrange the contraction as row-parallel. In Self-Attention, heads can likewise be divided across ranks before an output projection requires reconciliation. This intra-layer approach is the central Megatron-LM idea for training models that do not fit on one accelerator [2].

Tensor Parallelism reduces parameter and some activation storage per rank, but it introduces frequent collectives within the critical path of each layer. Enlarging  $n_T$  also reduces local matrix dimensions, which can make matrix multiplications less efficient. For this reason, Tensor Parallel groups are commonly kept modest and mapped to ranks with the highest available communication bandwidth. The exact group size is a performance decision, not a property of the Transformer equations.

## 5 Pipeline Parallelism

Pipeline Parallelism partitions model depth. With  $n_P$  stages, stage  $j$  owns a consecutive subset of the  $L$  Transformer blocks, receives activations from stage  $j - 1$ , and sends its output activations to stage  $j + 1$ . Backpropagation sends the corresponding activation gradients in the reverse direction. Unlike Tensor Parallelism, which cooperates within one layer, Pipeline Parallelism assigns different layers to different ranks. GPipe formalized the idea of executing layer partitions on separate accelerators with a split batch schedule [3].

If the full local batch is sent through all stages as one unit, all but one stage wait much of the time. Splitting it into  $M$  microbatches permits a stage to process one microbatch while another stage processes a different microbatch. Under an idealized pipeline with balanced stages, the utilization is approximately

$$u_{\text{pipe}} \approx \frac{M}{M + n_P - 1}, \quad 1 - u_{\text{pipe}} \approx \frac{n_P - 1}{M + n_P - 1}. \quad (8)$$

The second expression is the pipeline-bubble fraction. More microbatches improve utilization by amortizing the fill and drain periods, but they can increase activation memory, scheduling complexity, or the delay before one optimizer step completes. Pipeline schedules such as one-forward-one-backward change the activation-memory and timing trade-off, but they do not remove the dependency boundaries between stages. Megatron-LM studies these schedules and the composition of pipeline, tensor, and data parallelism at large scale [4].

## 6 Sequence Parallelism

Sequence Parallelism partitions selected activation tensors along the sequence dimension across ranks that already participate in Tensor Parallelism. It is neither Data Parallelism nor a replacement for the attention computation described in Chapter 3. Its purpose is to avoid replicating activations for operations that act independently across token positions, such as dropout, residual operations, and normalization. When an operation needs the full representation in a different layout, the system uses the appropriate gather or reduce-scatter boundary.

This distinction matters because activation memory can dominate after parameter state has been divided. If a replicated activation has shape  $B_{\text{local}} \times T \times d$ , a sequence partition over  $n_T$  ranks can reduce the stored local slice toward  $B_{\text{local}} \times \left(\frac{T}{n_T}\right) \times d$  for compatible operations. The saving is conditional: attention, tensor layouts, and communication boundaries determine which tensors can remain partitioned. Korthikanti et al. combine Sequence Parallelism with Tensor Parallelism to reduce activation-recomputation pressure in large Transformers [5].

## 7 Sharding with ZeRO and FSDP

ZeRO changes the replication policy inside a Data Parallel group. Rather than treating parameters, gradients, and optimizer state as three fully replicated objects, it shards them progressively. For a group of  $n_D$  ranks, the steady-state state terms in Table 1 are a useful first approximation. Activation memory and temporary communication buffers are additional terms.

| Scheme                    | State placement in one Data Parallel group                    | Approximate persistent state per rank                       |
|---------------------------|---------------------------------------------------------------|-------------------------------------------------------------|
| DDP                       | Parameters, gradients, and optimizer state replicated.        | $P(b_\theta + b_g + b_{\text{opt}})$                        |
| ZeRO Stage 1              | Optimizer state sharded; parameters and gradients replicated. | $P\left(b_\theta + b_g + \frac{b_{\text{opt}}}{n_D}\right)$ |
| ZeRO Stage 2              | Optimizer state and gradients sharded; parameters replicated. | $P\left(b_\theta + \frac{b_g + b_{\text{opt}}}{n_D}\right)$ |
| ZeRO Stage 3 / full shard | Parameters, gradients, and optimizer state sharded.           | $\frac{P(b_\theta + b_g + b_{\text{opt}})}{n_D}$            |

Table 1: Persistent model-state accounting under ZeRO-style sharding. Full-shard methods temporarily materialize needed parameters for computation, so peak memory also includes gathered parameters, activations, and communication buffers.

ZeRO Stage 1 removes replicated optimizer state; Stage 2 also removes replicated gradients; Stage 3 shards parameters as well. The progression preserves Data Parallel training semantics while changing which rank owns each state element between collectives. Rajbhandari et al. introduced this staged state-partitioning approach to reduce model-state memory without requiring conventional model parallelism for every case [6].

Fully Sharded Data Parallel (FSDP) is a Data Parallel implementation of the full-shard idea. It does not mean that individual layer matrix multiplications are Tensor Parallel. Instead, a rank holds parameter shards at rest, all-gathers the full parameters of a wrapped module when that module must compute, and reduce-scatters or otherwise shards gradients after the corresponding backward work. A well-chosen wrapping boundary limits how much full parameter state is materialized at once. PyTorch

FSDP is an implementation family designed around this behavior and its communication-memory trade-offs [7].

FSDP therefore differs from ordinary replicated DDP in its state ownership and communication schedule. DDP retains full parameter, gradient, and optimizer copies on every rank and mainly all-reduces gradients. FSDP retains shards and introduces parameter materialization around module execution. Both are Data Parallel in the sense that different ranks work on different data, but their memory and collective contracts differ substantially.

## 8 Communication, Overlap, and Scaling Efficiency

The arithmetic work in Chapter 9 is only one component of a training step. A compact communication model for  $n_{\text{coll}}$  collectives with total payload  $q$  is

$$t_{\text{comm}} \approx n_{\text{coll}}\alpha + q\beta, \quad (9)$$

where  $\alpha$  is a latency-like cost per collective and  $\beta$  is an inverse-bandwidth-like cost per byte. The model is deliberately simple: real performance depends on the collective algorithm, placement, contention, and message shape. Its main lesson is durable. Many small collective operations can be latency-bound, whereas large gradient or parameter exchanges can be bandwidth-bound.

Backward propagation exposes gradients in reverse layer order. A DDP runtime can place ready gradients into buckets and start their all-reduces while backward computation continues through earlier layers. Tensor and FSDP implementations can similarly prefetch or overlap some communication. The useful step time is therefore not always  $t_{\text{compute}} + t_{\text{comm}}$ ; it is closer to compute plus the portion of communication that remains uncovered. Overlap improves utilization only when enough independent computation exists and when communication is initiated early enough.

For a fixed workload, let  $t_1$  be a comparable one-rank or lower-scale reference time and let  $t_W$  be the distributed time on  $W$  ranks. Strong-scaling efficiency is

$$\eta(W) = \frac{t_1}{Wt_W}. \quad (10)$$

An ideal value is one. Actual efficiency falls because of collective communication, pipeline bubbles, load imbalance, data stalls, synchronization waits, and smaller local matrix multiplications. Consequently, theoretical division of FLOPs by  $W$  is not an execution prediction. Measured throughput and memory headroom should decide whether an additional parallel dimension is beneficial.

## 9 Failure, Synchronization, and Practical Strategy

Distributed training is synchronized state evolution. A collective mismatch—one rank calls a collective that another rank does not call, or ranks use incompatible tensor shapes—can deadlock or corrupt progress. A slow or failed rank can stall every replica at a synchronization point. A skipped optimizer step due to numerical control in Chapter 8 must be agreed upon by the relevant process group; allowing only some replicas to advance destroys parameter consistency.

Checkpointing must preserve more than model tensors. A sharded checkpoint needs the parallel configuration, shard layout, optimizer shards, scheduler state, random-state conventions, valid-token counters, and data-sampler state needed to resume the same global objective. Restoring under a changed world size can be supported only when the checkpoint format and resharding procedure explicitly permit it. A collection of rank-local files without this metadata is not a reliable recovery artifact.

A practical parallelism strategy begins with the bottleneck. If a complete training state fits per rank, replicated DDP is the simplest baseline. If model state does not fit, ZeRO or FSDP reduces replication. If one Transformer layer is too large or requires faster intra-layer computation, introduce a small Tensor Parallel group. If depth partitions remain necessary, add Pipeline Parallelism and choose enough microbatches to control Equation 8 without exhausting activation memory. Sequence Paral-

lelism becomes attractive when replicated activations remain the limiting term. The Megatron-LM results illustrate that Tensor, Pipeline, and Data Parallelism can be composed, but the group sizes must be chosen for the measured communication and memory regime rather than copied as a universal configuration [4].

## 10 Implementation Contracts

The parallelism contract must declare the world size, every process group, and the mapping from rank to Data, Tensor, Pipeline, and Sequence Parallel roles. It must state which tensors are replicated, which are sharded, their shard axes, and the collective that reconstructs or reduces each tensor. All members of a process group must execute collectives in the same order with compatible shapes. A test that uses a small deterministic batch should compare distributed logits, loss, selected gradients, and one optimizer update with a single-process reference within the expected numerical tolerance.

The objective contract preserves Chapter 5 exactly. It must define the microbatch-to-global-batch mapping in Equation 1, shard input examples without unintended duplication, and aggregate valid-token loss numerators and denominators consistently with Equation 3. Gradient accumulation, pipeline microbatches, and Data Parallel replicas are different clocks; the optimizer and learning-rate schedule from Chapter 7 advance once per declared global update, not once per local microbatch or rank.

The reliability contract coordinates finite-value detection, skipped updates, clipping, checkpoint barriers, and restart behavior across all relevant ranks. Monitoring should record per-rank and aggregate memory, collective wait time, communication volume, pipeline utilization, token throughput, update count, and valid-token count. These observables convert a slow or stalled job from an anecdote into a diagnosable execution trace.

## 11 Summary

Distributed training makes large-model pretraining feasible by separating two questions: how data, state, and computation are placed, and how ranks communicate the information needed for one coherent update. DDP replicates the model and averages gradients, which scales throughput but leaves replicated state as a memory limit. Tensor Parallelism splits layer computation; Pipeline Parallelism splits depth and pays bubble costs; Sequence Parallelism partitions compatible activation work. Their combination creates a multidimensional layout whose batch and world-size accounting must remain explicit.

ZeRO and FSDP reduce Data Parallel memory replication by sharding optimizer state, gradients, and eventually parameters, at the cost of additional communication and runtime complexity. The resulting system is not characterized by device count alone. Real scaling efficiency depends on collective cost, overlap, microbatching, load balance, memory peaks, and failure coordination. A correct training run is therefore one in which the distributed placement and synchronization rules preserve the same objective and optimizer trajectory intended by the single-process equations.

## References

- [1] S. Li *et al.*, “PyTorch Distributed: Experiences on Accelerating Data Parallel Training,” *arXiv preprint arXiv:2006.15704*, 2020, doi: 10.48550/arXiv.2006.15704.
- [2] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism,” *arXiv preprint arXiv:1909.08053*, 2019, doi: 10.48550/arXiv.1909.08053.
- [3] Y. Huang *et al.*, “GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism,” in *Advances in Neural Information Processing Systems 32*, Curran Associates, Inc., 2019. doi: 10.48550/arXiv.1811.06965.
- [4] D. Narayanan *et al.*, “Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021. doi: 10.1145/3458817.3476209.

- [5] V. A. Korthikanti *et al.*, “Reducing Activation Recomputation in Large Transformer Models,” in *Proceedings of Machine Learning and Systems*, MLSys, 2023. [Online]. Available: [https://proceedings.mlsys.org/paper\\_files/paper/2023/hash/80083951326cf5b35e5100260d64ed81-Abstract-mlsys2023.html](https://proceedings.mlsys.org/paper_files/paper/2023/hash/80083951326cf5b35e5100260d64ed81-Abstract-mlsys2023.html)
- [6] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020. doi: 10.1109/SC41405.2020.00024.
- [7] Y. Zhao *et al.*, “PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel,” *Proceedings of the VLDB Endowment*, vol. 16, no. 12, pp. 3848–3860, 2023, doi: 10.14778/3611540.3611569.